

LAPORAN PRAKTIKUM STRUKTUR DATA
IMPLEMENTASI QUEUE PADA BAHASA PEMROGRAMAN JAVA



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T
Asisten Praktikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat tuhan yang maha esa karena atas rahmat dan karunianya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari kegiatan praktikum Struktur Data dan Algoritma yang membahas implementasi struktur data Queue (antrian) dalam pemrograman Java. Materi yang dibahas meliputi Queue berbasis array, Queue berbasis linkedlist, proses pembalikan data queue menggunakan stack, serta iterasi elemen queue.

Melalui praktikum ini, penulis mempelajari konsep dasar queue sebagai dasar data dengan prinsip FIFO (First In First Out), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Implementasi dilakukan melalui berbagai pendekatan agar mahasiswa dapat memahami cara kerja queue secara lebih mendalam, baik secara manual menggunakan array maupun menggunakan library bawaan Java.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Padang, 28 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Dasar Teori	6
2.2 Langkah Kerja.....	7
2.2.1 Kode QueueArray_2511532024	7
2.2.2 Penjelasan Kode QueueArray_2511532024.....	8
2.2.3 Kode QueueArrayDriver_2511532024.....	11
2.2.4 Penjelasan Kode QueueArrayDriver_2511532024	12
2.2.5 Kode QueueLinkedList_2511532024.....	14
2.2.6 Penjelasan Kode QueueLinkedList_2511532024	14
2.2.7 Kode ReverseData_2511532024.....	16
2.2.8 Penjelasan Kode ReverseData_2511532024	16
2.2.9 Kode IterasiQueue_2511532024	18
2.2.10 Penjelasan Kode IterasiQueue_2511532024.....	18
2.2.11 Output Kode Program.....	19
BAB III PENUTUP	20
3.1 Kesimpulan	20
DAFTAR PUSTAKA	21
TUGAS PRAKTIKUM STRUKTUR DATA	22
4.1 Soal Tugas	22
4.2 Kode program.....	22
4.2.1 Class Queue_2511532024	22
4.2.2 Class AntrianLoket_2511532024	24
4.3 Penjelasan.....	25
4.3.1 Class Queue_2511532024	25
4.3.2 Class AntrianLoket_2511532024	29
4.4 Output Kode Program	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data secara efisien. Salah satu struktur data dasar yang sering digunakan adalah queue (antrian), yaitu struktur data yang bekerja berdasarkan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data yang pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrean pelanggan, proses printer, dan penjadwalan sistem komputer.

Dalam pemrograman java, queue dapat diimplementasikan menggunakan berbagai metode, seperti array, linked list, maupun kombinasi dengan struktur data lain seperti stack. Pemahaman terhadap berbagai implementasi ini penting agar mahasiswa dapat mengetahui kelebihan, kekurangan, dan penggunaan masing-masing metode sesuai kebutuhan program.

Praktikum ini bertujuan untuk mempelajari implementasi queue melalui beberapa pendekatan, yaitu queue berbasis array, queue berbasis linked list, reserve data menggunakan stack, dan iterasi data queue. Dengan mempelajari materi ini, mahasiswa diharapkan mampu memahami konsep queue secara teoritis maupun praktis, sehingga dapat menerapkannya dalam penyelesaian berbagai permasalahan pemrograman secara lebih efektif.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data queue dengan prinsip FIFO (First In First Out).
2. Mempelajari implementasi queue menggunakan array dan linked list dalam java.

3. Mengetahui penggunaan operasi dasar queue seperti enqueue, dequeue, peek, dan iterasi.
4. Memahami penerapan queue dalam berbagai kasus pemrograman, termasuk reverse data menggunakan stack.
5. Melatih kemampuan dalam merancang program berbasis struktur data secara sistematis.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Menambah pemahaman tentang pengelolaan data menggunakan struktur queue.
2. Meningkatkan keterampilan pemrograman java dalam implementasi struktur data.
3. Mengembangkan logika berpikir dan analisis algoritma.
4. Memberikan pengalaman praktis dalam menyelesaikan masalah menggunakan queue.
5. Menjadi dasar pembelajaran untuk struktur data yang lebih kompleks.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Queue atau antrian merupakan salah satu struktur dasar dalam ilmu komputer yang digunakan untuk menyimpan dan mengelola data secara berurutan berdasarkan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Konsep ini serupa dengan antrean di dunia nyata, seperti antrean pelanggan di kasir atau kendaraan di gerbang tol. Dalam queue, penambahan data dilakukan pada bagian belakang (rear), sedangkan penghapusan data dilakukan dari bagian depan (front).

Operasi dasar pada Queue meliputi enqueue untuk menambahkan elemen ke belakang antrian, dequeue untuk menghapus elemen dari depan antrian, peek untuk melihat elemen terdepan tanpa menghapusnya, isEmpty untuk memeriksa apakah antrian kosong, dan isFull untuk memeriksa apakah kapasitas antrian telah penuh. Operasi-operasi ini menjadikan Queue efisien dalam pengelolaan aliran data secara sistematis.

Implementasi Queue dapat dilakukan menggunakan array maupun linked list. Queue berbasis array lebih sederhana, namun memiliki keterbatasan kapasitas tetap dan dapat menjadi kurang efisien jika pengelolaan indeks tidak optimal. Sementara itu, Queue berbasis linked list lebih fleksibel karena ukuran dapat menyesuaikan kebutuhan data secara dinamis. Oleh sebab itu, linked list sering digunakan dalam implementasi Queue modern.

Dalam penerapan pemrograman, Queue digunakan pada berbagai sistem seperti penjadwalan proses dalam sistem operasi, manajemen antrean printer, komunikasi jaringan, simulasi pelayanan pelanggan, dan berbagai proses komputasi berurutan lainnya. Queue juga dapat dikombinasikan dengan stack untuk menyelesaikan operasi tertentu, seperti membalik urutan data dalam antrian (reverse queue).

2.2 Langkah Kerja

2.2.1 Kode QueueArray_2511532024

```
1. public class QueueArray_2511532024 {
2.     int front_2024, rear_2024, size_2024;
3.     int capacity_2024;
4.     int array_2024[];
5.
6.     public QueueArray_2511532024(int capacity) {
7.         this.capacity_2024 = capacity;
8.         front_2024 = this.size_2024 = 0;
9.         rear_2024 = capacity - 1;
10.        array_2024 = new int [this.capacity_2024];
11.    }
12.
13.    boolean isFull_2024(QueueArray_2511532024 queue) {
14.        return (queue.size_2024 ==
15.            queue.capacity_2024);
16.    }
17.    boolean isEmpty_2024(QueueArray_2511532024 queue) {
18.        return (queue.size_2024 == 0);
19.    }
20.
21.    void enqueue_2024 (int item) {
22.        if (isFull_2024(this))
23.            return;
24.        this.rear_2024 = (this.rear_2024 + 1) %
25.            this.capacity_2024;
26.        this.array_2024[this.rear_2024] = item;
27.        this.size_2024 = this.size_2024 + 1;
28.        System.out.println(item + " enqueued to
29.            queue");
30.    }
31.
32.    int dequeue_2024() {
33.        if (isEmpty_2024(this))
34.            return Integer.MIN_VALUE;
35.        int item = this.array_2024[this.front_2024];
36.        this.front_2024 = (this.front_2024 + 1) %
37.            this.capacity_2024;
38.        this.size_2024 = this.size_2024 - 1;
39.        return item;
40.    }
41.
42.    int front_2024() {
43.        if (isEmpty_2024(this))
44.            return Integer.MIN_VALUE;
45.        return this.array_2024[this.front_2024];
46.    }
47.
48.    int rear_2024() {
49.        if (isEmpty_2024(this))
50.            return Integer.MIN_VALUE;
51.        return this.array_2024[this.rear_2024];
52.    }
53. }
```

```

50.
51. // mencetak elemen antrian
52. void display_2024() {
53.     int i_2024;
54.     if(front_2024 == rear_2024) {
55.         System.out.printf("\nAntrian Kosong\n");
56.         return;
57.     }
58.
59.     // kunjungi dari belakang dan cetak
60.     for (i_2024 = front_2024; i_2024 < rear_2024;
        i_2024++) {
61.         System.out.printf(" %d <-- ",
            array_2024[i_2024]);
62.     }
63.     return;
64. }
65. }

```

2.2.2 Penjelasan Kode QueueArray_2511532024

1. Deklarasi kelas

```
public class QueueArray_2511532024 {
```

Baris ini mendefinisikan kelas QueueArray_2511532024 yang digunakan untuk mengimplementasikan struktur data Queue menggunakan array.

2. Deklarasi Variabel

```
int front_2024, rear_2024, size_2024;
int capacity_2024;
int array_2024[];
```

Variabel front_2024 menunjukkan posisi elemen terdepan, rear_2024 menunjukkan posisi elemen paling belakang, size_2024 menyimpan jumlah elemen dalam queue, capacity_2024 menyimpan kapasitas maksimum queue, dan array_2024 digunakan sebagai media penyimpanan data.

3. Konstruktor

```
public QueueArray_2511532024(int capacity) {
    this.capacity_2024 = capacity;
    front_2024 = this.size_2024 = 0;
```



```
rear_2024 = capacity - 1;
array_2024 = new int[this.capacity_2024];
}
```

Konstruktor ini digunakan untuk menginisialisasi queue dengan kapasitas tertentu. Posisi front dimulai dari 0, ukuran queue diatur 0, dan rear diatur ke indeks terakhir agar enqueue pertama dapat berjalan dengan sistem circular queue.

4. Method isFull dan isEmpty

```
boolean isFull_2024(QueueArray_2511532024 queue) {
    return (queue.size_2024 == queue.capacity_2024);
}

boolean isEmpty_2024(QueueArray_2511532024 queue) {
    return (queue.size_2024 == 0);
}
```

Method isFull_2024 digunakan untuk memeriksa apakah queue sudah penuh, sedangkan isEmpty_2024 digunakan untuk memeriksa apakah queue kosong.

5. Method Enqueue

```
void enqueue_2024(int item) {
    if (isFull_2024(this))
        return;
    this.rear_2024 = (this.rear_2024 + 1) %
this.capacity_2024;
    this.array_2024[this.rear_2024] = item;
    this.size_2024 = this.size_2024 + 1;
    System.out.println(item + " enqueued to queue");
}
```

Method ini digunakan untuk menambahkan elemen ke bagian belakang queue. Jika queue penuh, proses dibatalkan. Sistem circular queue diterapkan dengan operator modulo (%) agar indeks kembali ke awal ketika mencapai batas akhir array.

6. Method Dequeue

```
int dequeue_2024() {  
    if (isEmpty_2024(this))  
        return Integer.MIN_VALUE;  
    int item = this.array_2024[this.front_2024];  
    this.front_2024 = (this.front_2024 + 1) %  
this.capacity_2024;  
    this.size_2024 = this.size_2024 - 1;  
    return item;  
}
```

Method ini digunakan untuk menghapus elemen dari bagian depan queue. Jika queue kosong, akan mengembalikan Integer.MIN_VALUE sebagai penanda error.

7. Method Front dan Rear

```
int front_2024() {  
    if (isEmpty_2024(this))  
        return Integer.MIN_VALUE;  
    return this.array_2024[this.front_2024];  
}  
  
int rear_2024() {  
    if (isEmpty_2024(this))  
        return Integer.MIN_VALUE;  
    return this.array_2024[this.rear_2024];  
}
```

Method `front_2024()` digunakan untuk melihat elemen terdepan, sedangkan `rear_2024()` digunakan untuk melihat elemen paling belakang tanpa menghapus data.

8. Method Display

```
void display_2024() {
    int i_2024;
    if(front_2024 == rear_2024) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    for (i_2024 = front_2024; i_2024 < rear_2024; i_2024++)
    {
        System.out.printf(" %d <-- ", array_2024[i_2024]);
    }
    return;
}
```

Method ini digunakan untuk menampilkan seluruh elemen queue dari depan ke belakang. Jika queue kosong, program menampilkan pesan bahwa antrian kosong.

2.2.3 Kode QueueArrayDriver_2511532024

```
1. public class QueueArrayDriver_2511532024 {
2.
3.     public static void main(String[] args) {
4.         QueueArray_2511532024 queue_2024 = new
         QueueArray_2511532024(1000);
5.         queue_2024.enqueue_2024(10);
6.         queue_2024.enqueue_2024(20);
7.         queue_2024.enqueue_2024(30);
8.         queue_2024.enqueue_2024(40);
9.         System.out.println("Item di depan " +
         queue_2024.front_2024());
10.        System.out.println("Item paling belakang " +
         queue_2024.rear_2024());
11.        System.out.println("tampilan queue");
12.        queue_2024.display_2024();
13.        System.out.println();
14.        System.out.println(queue_2024.dequeue_2024()
         + " dihapus dari queue");
}
```

```

15.     System.out.println("Item di depan: " +
        queue_2024.front_2024());
16.     System.out.println("Item di belakang: " +
        queue_2024.rear_2024());
17.     System.out.println("tampilan queue setelah
        saru data dihapus");
18.     queue_2024.display_2024();
19. }
20. }

```

2.2.4 Penjelasan Kode QueueArrayDriver_2511532024

1. Deklarasi kelas dan method main

```

public class QueueArrayDriver_2511532024 {
    public static void main(String[] args) {

```

Baris ini mendefinisikan kelas driver QueueArrayDriver_2511532024 sebagai program utama untuk menjalankan dan menguji implementasi Queue berbasis array.

2. Pembuatan objek queue

```

QueueArray_2511532024 queue_2024 = new
QueueArray_2511532024(1000);

```

Objek queue_2024 dibuat dengan kapasitas maksimum 1000 elemen. Ini berarti queue mampu menampung hingga 1000 data integer.

3. Menambahkan data ke queue

```

queue_2024.enqueue_2024(10);
queue_2024.enqueue_2024(20);
queue_2024.enqueue_2024(30);
queue_2024.enqueue_2024(40);

```

Keempat baris ini digunakan untuk menambahkan data ke dalam queue menggunakan operasi enqueue, sehingga data masuk secara berurutan dari belakang antrian.

4. Menampilkan elemen depan dan belakang

```
System.out.println("Item      di      depan      "      +  
queue_2024.front_2024());  
System.out.println("Item      paling      belakang      "      +  
queue_2024.rear_2024());
```

Method `front_2024()` digunakan untuk melihat elemen terdepan, sedangkan `rear_2024()` digunakan untuk melihat elemen paling belakang.

5. Menampilkan isi queue

```
System.out.println("tampilan queue");  
queue_2024.display_2024();
```

Bagian ini menampilkan seluruh isi queue dari depan ke belakang menggunakan method `display_2024()`.

6. Menghapus data dari queue

```
System.out.println(queue_2024.dequeue_2024() + " dihapus  
dari queue");
```

Method `dequeue_2024()` digunakan untuk menghapus elemen terdepan dari queue sesuai prinsip FIFO.

7. Menampilkan kondisi queue setelah dequeue

```
System.out.println("Item      di      depan:      "      +  
queue_2024.front_2024());  
System.out.println("Item      di      belakang:      "      +  
queue_2024.rear_2024());  
System.out.println("tampilan queue setelah satu data  
dihapus");  
queue_2024.display_2024();
```

Setelah satu elemen dihapus, program menampilkan kembali elemen terdepan, elemen paling belakang, dan seluruh isi queue terbaru.

2.2.5 Kode QueueLinkedList_2511532024

```
1. public class QueueLinkedList_2511532024 {
2.
3.     public static void main(String[] args) {
4.         Queue<Integer> q_2024 = new LinkedList<>();
5.
6.         // tambah elemen {0, 1, 2, 3, 4, 5} ke
        antrian
7.         for (int i_2024 = 0; i_2024 < 6; i_2024++)
8.             q_2024.add(i_2024);
9.
10.        // menampilkan isi antrian
11.        System.out.println("Elemen Antrian " +
            q_2024);
12.
13.        // untuk menghapus kepala antrian
14.        int hapus_2024 = q_2024.remove();
15.        System.out.println("Hapus elemen = " +
            hapus_2024);
16.        System.out.println(q_2024);
17.
18.        // untuk melihat antrian terdepan
19.        int depan_2024 = q_2024.peek();
20.        System.out.println("Kepala Antrian = " +
            depan_2024);
21.
22.        // untuk melihat jumlah antrian
23.        int banyak_2024 = q_2024.size();
24.        System.out.println("Size Antrian = " +
            banyak_2024);
25.    }
26. }
```

2.2.6 Penjelasan Kode QueueLinkedList_2511532024

1. Deklarasi kelas dan method

```
public class QueueLinkedList_2511532024 {
    public static void main(String[] args) {
```

Baris ini mendefinisikan kelas QueueLinkedList_2511532024 sebagai program utama untuk mengimplementasikan Queue menggunakan struktur LinkedList.

2. Pembuatan objek queue

```
Queue<Integer> q_2024 = new LinkedList<>();
```

Objek q_2024 dibuat menggunakan interface Queue dengan implementasi LinkedList, sehingga queue dapat bertambah

dan berkurang secara dinamis tanpa batas kapasitas tetap seperti array.

3. Menambahkan elemen ke queue

```
for (int i_2024 = 0; i_2024 < 6; i_2024++)  
    q_2024.add(i_2024);
```

Perulangan ini digunakan untuk menambahkan elemen 0 hingga 5 ke dalam queue secara berurutan menggunakan method add().

4. Menampilkan isi queue

```
System.out.println("Elemen Antrian " + q_2024);
```

Menampilkan seluruh elemen queue yang tersimpan.

5. Menghapus elemen terdepan

```
int hapus_2024 = q_2024.remove();  
System.out.println("Hapus elemen = " + hapus_2024);  
System.out.println(q_2024);
```

Method remove() digunakan untuk menghapus elemen paling depan dari queue sesuai prinsip FIFO, kemudian hasil penghapusan ditampilkan.

6. Melihat elemen terdepan

```
int depan_2024 = q_2024.peek();  
System.out.println("Kepala Antrian = " + depan_2024);
```

Method peek() digunakan untuk melihat elemen terdepan tanpa menghapusnya.

7. Menampilkan jumlah elemen

```
int banyak_2024 = q_2024.size();  
System.out.println("Size Antrian = " + banyak_2024);
```

Method size() digunakan untuk mengetahui jumlah elemen yang tersisa dalam queue.

2.2.7 Kode ReverseData_2511532024

```
1. public class ReverseData_2511532024 {
2.
3.     public static void main(String[] args) {
4.         Queue<Integer> q_2024 = new
           LinkedList<Integer>();
5.         q_2024.add(1);
6.         q_2024.add(2);
7.         q_2024.add(3); // {1,2,3}
8.         System.out.println("sebelum reverse = " +
           q_2024);
9.         Stack<Integer> s_2024 = new Stack<Integer>();
10.        while (!q_2024.isEmpty()) { // Q -> S
11.            s_2024.push(q_2024.remove());
12.        }
13.        while (!s_2024.isEmpty()) { // S -> Q
14.            q_2024.add(s_2024.pop());
15.        }
16.        System.out.println("sesudah reverse = " +
           q_2024); // {3,2,1}
17.    }
18. }
```

2.2.8 Penjelasan Kode ReverseData_2511532024

1. Deklarasi kelas dan method main

```
public class ReverseData_2511532024 {
    public static void main(String[] args) {
```

Baris ini mendefinisikan kelas ReverseData_2511532024 sebagai program utama untuk melakukan pembalikan urutan data pada queue menggunakan bantuan stack.

2. Pembuatan queue dan penambahan data

```
Queue<Integer> q_2024 = new LinkedList<Integer>();
q_2024.add(1);
q_2024.add(2);
q_2024.add(3);
```

Queue dibuat menggunakan LinkedList, kemudian elemen 1, 2, dan 3 dimasukkan secara berurutan ke dalam antrian.

3. Menampilkan queue sebelum reverse

```
System.out.println("sebelum reverse = " + q_2024);
```

Menampilkan isi queue sebelum proses pembalikan data dilakukan.

4. Pembuatan stack

```
Stack<Integer> s_2024 = new Stack<Integer>();
```

Stack digunakan sebagai struktur bantu karena sifat LIFO (Last In First Out) sangat cocok untuk membalik urutan data.

5. Memindahkan data dari queue ke stack

```
while (!q_2024.isEmpty()) {  
    s_2024.push(q_2024.remove());  
}
```

Seluruh elemen queue dipindahkan ke stack satu per satu. Karena stack bekerja dengan prinsip LIFO, urutan data otomatis terbalik.

6. Memindahkan data dari stack ke queue

```
while (!s_2024.isEmpty()) {  
    q_2024.add(s_2024.pop());  
}
```

Elemen dari stack dimasukkan kembali ke queue, sehingga queue kini memiliki urutan terbalik dari data awal.

7. Menampilkan queue setelah reverse

```
System.out.println("sesudah reverse = " + q_2024);
```

Menampilkan hasil akhir queue setelah proses pembalikan selesai.

2.2.9 Kode IterasiQueue_2511532024

```
1. public class IterasiQueue_2511532024 {  
2.  
3.     public static void main(String[] args) {  
4.         Queue<String> q_2024 = new LinkedList<>();  
5.         q_2024.add("Praktikum");  
6.         q_2024.add("Struktur");  
7.         q_2024.add("Data");  
8.         q_2024.add("Dan");  
9.         q_2024.add("Algoritma");  
10.        Iterator<String> iterator = q_2024.iterator();  
11.        while (iterator.hasNext()) {  
12.            System.out.print(iterator.next() + " ");  
13.        }  
14.    }  
15. }
```

2.2.10 Penjelasan Kode IterasiQueue_2511532024

1. Deklarasi kelas dan method main

```
public class IterasiQueue_2511532024 {  
    public static void main(String[] args) {
```

Baris ini mendefinisikan kelas IterasiQueue_2511532024 sebagai program utama untuk menampilkan elemen-elemen queue menggunakan proses iterasi.

2. Pembuatan queue

```
Queue<String> q_2024 = new LinkedList<>();
```

Queue bertipe String dibuat menggunakan implementasi LinkedList, sehingga data teks dapat disimpan secara dinamis.

3. Menambahkan data ke queue

```
q_2024.add("Praktikum");  
q_2024.add("Struktur");  
q_2024.add("Data");  
q_2024.add("Dan");  
q_2024.add("Algoritma");
```

Data berupa kata-kata dimasukkan ke dalam queue secara berurutan hingga membentuk kalimat.

4. Membuat iterator

```
Iterator<String> iterator = q_2024.iterator();
```

Objek Iterator digunakan untuk menelusuri seluruh elemen queue satu per satu tanpa menghapus data.

5. Proses iterasi

```
while (iterator.hasNext()) {  
    System.out.print(iterator.next() + " ");  
}
```

Perulangan while akan terus berjalan selama masih ada elemen berikutnya. Method next() mengambil elemen satu per satu dan menampilkannya.

2.2.11 Output Kode Program

1. Output Kode Program QueueArrayDriver_2511532024

```
10 enqueued to queue  
20 enqueued to queue  
30 enqueued to queue  
40 enqueued to queue  
Item di depan 10  
Item paling belakang 40  
tampilan queue  
10 <-- 20 <-- 30 <--  
10 dihapus dari queue  
Item di depan: 20  
Item di belakang: 40  
tampilan queue setelah satu data dihapus  
20 <-- 30 <--
```

2. Output Kode Program QueueLinkedList_2511532024

```
Elemen Antrian [0, 1, 2, 3, 4, 5]  
Hapus elemen = 0  
[1, 2, 3, 4, 5]  
Kepala Antrian = 1  
Size Antrian = 5
```

3. Output Kode Program ReverseData_2511532024

```
sebelum reverse = [1, 2, 3]  
sesudah reverse = [3, 2, 1]
```

4. Output Kode Program IterasiQueue_2511532024

```
Praktikum Struktur Data Dan Algoritma
```

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue merupakan metode penyimpanan data yang bekerja berdasarkan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan keluar lebih dahulu. Implementasi Queue dapat dilakukan menggunakan array maupun linked list, dengan masing-masing memiliki karakteristik tersendiri. Selain itu, Queue juga dapat dikombinasikan dengan struktur data lain seperti stack untuk menyelesaikan proses tertentu, seperti pembalikan data.

Melalui berbagai latihan program yang telah dibuat, praktikan dapat memahami operasi dasar Queue seperti enqueue, dequeue, peek, iterasi, serta penerapannya dalam berbagai kasus pemrograman. Pemahaman ini memberikan dasar yang kuat dalam penggunaan struktur data secara efektif untuk pengembangan program yang lebih terstruktur dan efisien.

DAFTAR PUSTAKA

- [1] Wahyudi, Dr., S.T., M.T., *Struktur Data Queue*, Padang: Departemen Teknik Komputer, Fakultas Teknologi Informasi, Universitas Andalas, 2026.
- [2] Wahyudi, Dr., S.T., M.T., *Modul Praktikum Struktur Data dan Algoritma*, Padang: Universitas Andalas, 2026.

TUGAS PRAKTIKUM STRUKTUR DATA

4.1 Soal Tugas

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian) menggunakan array dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO (First In First Out).

4.2 Kode program

4.2.1 Class Queue_2511532024

```
1. import java.util.LinkedList;
2. import java.util.Queue;
3.
4. public class Queue_2511532024 {
5.
6.     int front_2024, rear_2024, size_2024;
7.     int max_2024;
8.     String queue_2024[];
9.
10.    // Konstruktor
11.    public Queue_2511532024(int max) {
12.        this.max_2024 = max;
13.        front_2024 = 0;
14.        size_2024 = 0;
15.        rear_2024 = max - 1;
16.        queue_2024 = new String[this.max_2024];
17.    }
18.
19.    boolean isFull_2024() {
20.        return (size_2024 == max_2024);
21.    }
22.
23.    boolean isEmpty_2024() {
24.        return (size_2024 == 0);
25.    }
26.
27.    void enqueue_2024(String nama_2024) {
28.        if (isFull_2024()) {
29.            System.out.println("Antrian penuh.");
30.            return;
31.        }
32.
33.        rear_2024 = (rear_2024 + 1) % max_2024;
34.        queue_2024[rear_2024] = nama_2024;
35.        size_2024++;
36.        System.out.println("Data berhasil ditambahkan ke antrian");
```

```

37. }
38.
39. String dequeue_2024() {
40.     if (isEmpty_2024()) {
41.         return null;
42.     }
43.
44.     String pelanggan_2024 =
        queue_2024[front_2024];
45.     front_2024 = (front_2024 + 1) % max_2024;
46.     size_2024--;
47.     return pelanggan_2024;
48. }
49.
50. void display_2024() {
51.     if (isEmpty_2024()) {
52.         System.out.println("\nAntrian kosong.");
53.         return;
54.     }
55.
56.     System.out.println("\nIsi antrian: ");
57.     for (int i_2024 = 0; i_2024 < size_2024;
        i_2024++) {
58.         int index_2024 = (front_2024 + i_2024) %
            max_2024;
59.         System.out.println((i_2024 + 1) + ". "
            + queue_2024[index_2024]);
60.     }
61. }
62.
63. void reverse_2024() {
64.     if (isEmpty_2024()) {
65.         System.out.println("\nAntrian kosong.");
66.         return;
67.     }
68.
69.     java.util.Queue<String> qTemp_2024 = new
        java.util.LinkedList<>();
70.     java.util.Stack<String> s_2024 = new
        java.util.Stack<>();
71.
72.     for (int i_2024 = 0; i_2024 < size_2024;
        i_2024++) {
73.         int index_2024 = (front_2024 + i_2024) %
            max_2024;
74.         qTemp_2024.add(queue_2024[index_2024]);
75.     }
76.
77.     while (!qTemp_2024.isEmpty()) {
78.         s_2024.push(qTemp_2024.remove());
79.     }
80.
81.     for (int i_2024 = 0; i_2024 < size_2024;
        i_2024++) {
82.         int index_2024 = (front_2024 + i_2024) %
            max_2024;
83.         queue_2024[index_2024] = s_2024.pop();

```



```

38.                 case 3:
39.                     q_2024.display_2024();
40.                     break;
41.
42.                 case 4:
43.                     q_2024.reverse_2024();
44.                     q_2024.display_2024();
45.                     break;
46.
47.                 case 5:
48.
49.                     System.out.println("\nProgram selesai");
50.                     break;
51.
52.                 default:
53.                     System.out.println("\nPilihan tidak valid.");
54.                 }
55.             } while (pilih_2024 != 5);
56.             sc_2024.close();
57.         }
58.     }

```

4.3 Penjelasan

4.3.1 Class Queue_2511532024

1. Deklarasi Variabel dan Struktur Queue

```

int front_2024, rear_2024, size_2024;
int max_2024;
String queue_2024[];

```

Baris ini digunakan untuk mendeklarasikan variabel utama dalam struktur queue. `front_2024` menunjukkan posisi elemen terdepan, `rear_2024` menunjukkan posisi elemen terakhir, `size_2024` menyimpan jumlah data dalam antrian, `max_2024` menentukan kapasitas maksimum queue, dan `queue_2024[]` adalah array bertipe String yang berfungsi sebagai tempat penyimpanan data antrian.

2. Konstruktor Queue

```

public Queue_2511532024(int max) {
    this.max_2024 = max;
    front_2024 = 0;
}

```

```
size_2024 = 0;
rear_2024 = max - 1;
queue_2024 = new String[this.max_2024];
}
```

Bagian ini merupakan konstruktor yang digunakan untuk menginisialisasi queue dengan kapasitas tertentu sesuai nilai max. Posisi depan dimulai dari indeks 0, ukuran queue diatur kosong (0), posisi rear diatur ke indeks terakhir array untuk mendukung circular queue, dan array queue dibuat sesuai kapasitas maksimum.

3. Method Pengecekan Queue

```
boolean isFull_2024() {
    return (size_2024 == max_2024);
}

boolean isEmpty_2024() {
    return (size_2024 == 0);
}
```

Method ini berfungsi untuk memeriksa kondisi queue. isFull_2024() digunakan untuk mengecek apakah antrian sudah penuh, sedangkan isEmpty_2024() digunakan untuk mengecek apakah antrian masih kosong.

4. Method Enqueue (Menambah Data)

```
void enqueue_2024(String nama_2024) {
    if (isFull_2024()) {
        System.out.println("Antrian penuh.");
        return;
    }

    rear_2024 = (rear_2024 + 1) % max_2024;
    queue_2024[rear_2024] = nama_2024;
}
```

```
size_2024++;  
System.out.println("Data berhasil ditambahkan ke antrian");  
}
```

Method ini digunakan untuk menambahkan data baru ke dalam queue. Jika antrian penuh, data tidak dapat ditambahkan. Jika masih tersedia ruang, posisi rear digeser secara melingkar menggunakan modulo, data dimasukkan ke posisi tersebut, lalu ukuran queue bertambah.

5. Method Dequeue (Menghapus Data)

```
String dequeue_2024() {  
    if (isEmpty_2024()) {  
        return null;  
    }  
  
    String pelanggan_2024 = queue_2024[front_2024];  
    front_2024 = (front_2024 + 1) % max_2024;  
    size_2024--;  
    return pelanggan_2024;  
}
```

Method ini berfungsi untuk mengambil dan menghapus data paling depan dari queue. Jika queue kosong, method mengembalikan null. Jika ada data, elemen depan diambil, posisi front digeser ke depan secara circular, dan ukuran queue dikurangi.

6. Method Display (Menampilkan Isi Queue)

```
void display_2024() {  
    if (isEmpty_2024()) {  
        System.out.println("\nAntrian kosong.");  
        return;  
    }  
}
```

```

System.out.println("\nIsi antrian: ");
for (int i_2024 = 0; i_2024 < size_2024; i_2024++) {
    int index_2024 = (front_2024 + i_2024) % max_2024;
    System.out.println((i_2024 + 1) + ". " +
queue_2024[index_2024]);
}
}

```

Method ini digunakan untuk menampilkan seluruh isi queue dari depan ke belakang. Jika queue kosong, program menampilkan pesan kosong. Jika terdapat data, perulangan digunakan untuk menampilkan setiap elemen sesuai urutan antrian.

7. Method Reverse (Membalik Queue)

```

void reverse_2024() {
    if (isEmpty_2024()) {
        System.out.println("\nAntrian kosong.");
        return;
    }
    java.util.Queue<String> qTemp_2024 = new
java.util.LinkedList<>();
    java.util.Stack<String> s_2024 = new java.util.Stack<>();
}

```

Bagian awal method ini digunakan untuk mengecek apakah queue kosong. Jika tidak kosong, dibuat queue sementara (LinkedList) dan stack untuk membantu proses pembalikan data.

```

for (int i_2024 = 0; i_2024 < size_2024; i_2024++) {
    int index_2024 = (front_2024 + i_2024) % max_2024;
    qTemp_2024.add(queue_2024[index_2024]);
}

```

Seluruh elemen queue dipindahkan ke queue sementara.

```

while (!qTemp_2024.isEmpty()) {
    s_2024.push(qTemp_2024.remove());
}

```

Data dari queue sementara dimasukkan ke stack agar urutannya terbalik.

```
for (int i_2024 = 0; i_2024 < size_2024; i_2024++) {  
    int index_2024 = (front_2024 + i_2024) % max_2024;  
    queue_2024[index_2024] = s_2024.pop();  
}
```

Data dari stack dikembalikan ke queue utama sehingga urutan antrian menjadi terbalik.

```
System.out.println("\nAntrian berhasil dibalik.");
```

Baris ini menampilkan pesan bahwa proses reverse berhasil dilakukan.

4.3.2 Class **AntrianLoket_2511532024**

1. Deklarasi Kelas dan Method Main

```
public class AntrianLoket_2511532024 {  
    public static void main(String[] args) {
```

Baris ini mendefinisikan kelas `AntrianLoket_2511532024` sebagai program utama yang menjalankan sistem antrian loket. Method `main` menjadi titik awal eksekusi program, tempat seluruh proses menu dan interaksi pengguna berlangsung. Karena tentu manusia lebih suka memilih menu berulang kali daripada hidup sederhana.

2. Inisialisasi Scanner, Queue, dan Variabel Menu

```
Scanner sc_2024 = new Scanner(System.in);  
Queue_2511532024 q_2024 = new Queue_2511532024(100);  
int pilih_2024;
```

Bagian ini digunakan untuk menyiapkan input pengguna melalui Scanner, membuat objek queue dengan kapasitas maksimal 100 data pelanggan, serta mendeklarasikan variabel `pilih_2024` untuk menyimpan pilihan menu pengguna.

3. Perulangan Menu Utama

```
do {  
    System.out.println("\n=== PROGRAM ANTRIAN LOKET  
===");  
    System.out.println("1. Tambah Antrian");  
    System.out.println("2. Hapus Antrian");  
    System.out.println("3. Tampilkan Antrian");  
    System.out.println("4. Reverse");  
    System.out.println("5. Keluar");  
    System.out.print("Pilih menu: ");  
    pilih_2024 = sc_2024.nextInt();  
    sc_2024.nextLine();  
}
```

Perulangan do-while digunakan agar menu terus tampil sampai pengguna memilih keluar. Program menampilkan daftar menu, menerima input pilihan, lalu `nextLine()` digunakan untuk membersihkan buffer setelah input angka agar tidak mengganggu input berikutnya. Detail kecil yang sering diabaikan, lalu program error, lalu panik.

4. Proses Pilihan Menu dengan Switch Case

```
switch (pilih_2024) {
```

Struktur switch-case digunakan untuk menjalankan tindakan berbeda berdasarkan pilihan menu pengguna.

5. Case 1 – Tambah Antrian

```
case 1:  
    System.out.print("\nMasukkan nama pelanggan: ");  
    String nama_2024 = sc_2024.nextLine();  
    q_2024.enqueue_2024(nama_2024);  
    break;
```

Pada menu ini, pengguna memasukkan nama pelanggan yang kemudian ditambahkan ke dalam queue menggunakan method `enqueue_2024()`. Data baru akan masuk ke posisi belakang antrian.

6. Case 2 – Hapus Antrian

case 2:

```
String hapus_2024 = q_2024.dequeue_2024();  
if (hapus_2024 == null) {  
    System.out.println("\nAntrian kosong.");  
} else {  
    System.out.println();  
    System.out.println(hapus_2024 + " telah dilayani");  
}  
break;
```

Menu ini digunakan untuk menghapus pelanggan terdepan dari antrian menggunakan method `dequeue_2024()`. Jika queue kosong, program menampilkan pesan kosong. Jika ada data, nama pelanggan yang dihapus akan ditampilkan sebagai pelanggan yang telah dilayani.

7. Case 3 – Tampilan Antrian

case 3:

```
q_2024.display_2024();  
break;
```

Pilihan ini menampilkan seluruh isi antrian saat ini menggunakan method `display_2024()`, sehingga pengguna dapat melihat urutan pelanggan.

8. Case 4 – Reverse Antrian

case 4:

```
q_2024.reverse_2024();  
q_2024.display_2024();  
break;
```

Menu ini membalik urutan antrian menggunakan method `reverse_2024()`, lalu langsung menampilkan hasil antrian terbaru. Fitur yang mungkin tidak realistis di loket nyata, tapi setidaknya algoritmanya menarik.

9. Case 5 – Keluar Program

case 5:

```
System.out.println("\nProgram selesai");  
break;
```

Jika pengguna memilih menu 5, program akan menampilkan pesan selesai dan keluar dari perulangan.

10. Default – Input Tidak Valid

default:

```
System.out.println("\nPilihan tidak valid.");
```

Bagian ini dijalankan jika pengguna memasukkan pilihan di luar daftar menu yang tersedia.

11. Kondisi Berhenti dan Menutup Scanner

```
} while (pilih_2024 != 5);  
sc_2024.close();
```

Perulangan akan terus berjalan selama pengguna tidak memilih menu 5. Setelah program selesai, Scanner ditutup untuk mengakhiri penggunaan sumber input secara aman. Sebab bahkan program pun tahu kapan harus berhenti, sesuatu yang kadang sulit dilakukan manusia.

4.4 Output Kode Program

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: nisa
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: karina
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: irene
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: ariana
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4

Antrian berhasil dibalik.

Isi antrian:
1. ariana
2. irene
3. karina
4. nisa

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2

ariana telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET ===
```

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

```
Pilih menu: 3
```

```
Isi antrian:
```

1. irene
2. karina
3. nisa

```
=== PROGRAM ANTRIAN LOKET ===
```

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

```
Pilih menu: 5
```

```
|
```

```
Program selesai
```